

UNIVA Module System & Installer Engine

Specification & Developer Guide

UNIVA Core Architecture Team

UNIVA Systems

Table of Contents

1. UNIVA Module System & Installer Engine — Specification Guide	3
1.1 Executive Summary & Quick Navigation	4
1.2 Core System Architecture Blueprint	5
1.3 Command Line Quick Reference	5
2. Architecture & Core	6
2.1 1. Executive Overview & Architectural Core Principles	6
2.2 2. The Two Fundamental Module Types	10
2.3 3. The Five-Engine Installer Orchestration Architecture	12
2.4 4. Complete Declarative Module Folder Structure	14
3. Five-Engine Specifications	18
3.1 5. Engine 1 — Manifest Engine & Specification (<code>manifest.json</code>)	19
3.2 6. Engine 2 — Database Engine & Declarative Schemas (<code>models/*.json</code>)	23
3.3 7. Engine 3 — Integration Engine (Data Linking & Function Contracts)	31
3.4 8. Engine 4 — Security Engine & RBAC/ABAC Declarations (<code>security/access.json</code>)	35
3.5 9. Engine 5 — UI Engine & Live View Registration (<code>views/</code>)	40
4. Operations & Mechanics	44
4.1 10. Lifecycle Event Hooks & Migrations (<code>hooks/</code> & <code>migrations/</code>)	44
4.2 11. System Architecture & Data Flow (Mermaid Diagrams)	46
4.3 12. Complete Worked Module Case Studies	49
5. Reference & Guides	53
5.1 13. REST API Endpoint Reference Guide	53
5.2 14. Developer Workflow, Testing & Troubleshooting	55
5.3 15. Quick Reference Cheatsheet & Terminal Commands	57
6. Deep Dives & Gaps	58
6.1 16. Architecture & Security Critique: Gap Analysis	58

1. UNIVA Module System & Installer Engine — Specification Guide

Architectural Premise: The UNIVA Installer Engine is a 100% data-driven orchestration system comprising **Five Engines, One Orchestrator**. Modules never ship executable backend server code except sandboxed hooks. Any cross-module logic reuse or data referencing is mediated via declared, versioned function contracts (`integrations/uses.json` and `functions/exposes.json`). Frontend views dynamically render live from metadata without app rebuilds or full-page swaps; custom UI widgets execute exclusively inside sandboxed `<iframe>` containers via the `postMessage` protocol.

1.1 Executive Summary & Quick Navigation

Welcome to the **UNIVA Module System & Installer Engine Single Source of Truth**. This documentation suite defines the architecture, database schema materialization, security matrices, UI live rendering, lifecycle hooks, case studies, and critical security gap analysis for building and maintaining UNIVA modules.

CHAPTER	TOPIC	HIGHLIGHTS
1. Architecture Overview	Declarative Vision & Security	100% Declarative model, Zero-Preinstalled-Tables, SQL Injection defense
2. Module Types	Main vs Extension Modules	Feature comparison matrix, structural rules
3. 5-Engine Orchestration	Pipeline Execution	Step-by-step 10-stage install pipeline, rollback & concurrency locks
4. Directory Layout	Module Directory Structure	File inventory, mandatory vs optional checklists
Engine 1 — Manifest	<code>manifest.json</code> Spec	JSON Schema, SemVer, prerequisites & integration contracts
Engine 2 — Database	<code>models/*.json</code> Schemas	DDL materialization, system columns, soft-delete unique indexes
Engine 3 — Integration	Data Linking & Function Contracts	Two-phase FK resolution, <code>sys_field_relation</code> , <code>FunctionRegistry</code>
Engine 4 — Security	<code>security/access.json</code> Rules	RBAC CRUD matrix, ABAC <code>domain_filter</code> row rules, role seeding
Engine 5 — UI Engine	<code>views/*.json</code> Specifications	Dynamic rendering, column widgets, sandboxed IFrame <code>postMessage</code>
Lifecycle & Migrations	Sandboxed Hooks & Migrations	Process isolation ceiling (5s, 128MB), versioned DDL migrations
System Visual Diagrams	Mermaid Sequence & Flow Charts	Module install flow, UI view composition, cross-module calls
Worked Case Studies	Complete Worked Modules	Code & manifest specs for <code>farms</code> , <code>invoicing</code> , <code>schedule</code> , <code>pdf_exporter</code>
REST API Reference	Endpoint Specifications	Installer APIs & <code>/api/data/{model_name}</code> CRUD endpoints
Developer Workflow	Step-by-Step Guide	Authoring walkthrough, local debugging, error remediation guide
Cheatsheet & CLI	Terminal Commands	Docker control, JSON validation scripts, PowerShell/Curl commands
Gap Analysis & Critique	Architectural & Security Audit	10 priority security gaps, installer bugs, missing engines, permission fix

1.2 Core System Architecture Blueprint

```

                                flowchart TD
                                Core(["<i class='fa-solid fa-gears'></i> InstallerEngine.install(account_id,
                                                                module_name)"])

                                Core --> E1(["<i class='fa-solid fa-file-code'></i> Engine 1: Manifest Loader<br/>SemVer &
                                                                prerequisite validation"])
                                Core --> E2(["<i class='fa-solid fa-database'></i> Engine 2: Database Engine<br/>DDL
                                                                Creation & Catalog"])
                                Core --> E3(["<i class='fa-solid fa-link'></i> Engine 3: Integration Engine<br/>FK
                                                                Constraints & Functions"])
                                Core --> E4(["<i class='fa-solid fa-user-lock'></i> Engine 4: Security Engine<br/>RBAC &
                                                                ABAC Matrix"])
                                Core --> E5(["<i class='fa-solid fa-palette'></i> Engine 5: UI Engine<br/>View Schemas &
                                                                Asset Manifests"])

                                style Core fill:#0f172a,stroke:#38bdf8,stroke-width:3px,color:#ffffff
                                style E1 fill:#eff6ff,stroke:#3b82f6,stroke-width:2px,color:#1e3a8a
                                style E2 fill:#f0f9ff,stroke:#0284c7,stroke-width:2px,color:#0369a1
                                style E3 fill:#f0fdf4,stroke:#16a34a,stroke-width:2px,color:#15803d
                                style E4 fill:#fffbeb,stroke:#d97706,stroke-width:2px,color:#78350f
                                style E5 fill:#faf5ff,stroke:#9333ea,stroke-width:2px,color:#6b21a8

```

1.3 Command Line Quick Reference

```

# Command line to serve MkDocs documentation locally
cd d:\Work\UNIVA\PLUGIN-MODULE-SYSTEM\mkdoc
mkdocs serve

# Command line to build static documentation site
mkdocs build

```

2. Architecture & Core

2.1 1. Executive Overview & Architectural Core Principles

2.1.1 1.1 The 100% Declarative Engine Vision

The UNIVA Module System solves the classic enterprise extension dilemma: how to allow third-party developers, custom domain builders, or tenant organizations to extend software capabilities **without introducing arbitrary code execution vulnerabilities, platform instability, or complex deployment rebuild cycles**.

In traditional monolithic extensions (such as unvetted Python plugins), plugins execute arbitrary server code in the host process. If a plugin crashes, memory leaks, or contains security exploits, the entire host application suffers.

UNIVA replaces arbitrary plugin code with a **100% Declarative Engine Architecture**. Modules do not supply executable backend code; instead, they supply structured JSON manifests, database field specifications, security matrices, UI view definitions, and contract declarations. The **UNIVA Installer Engine** reads, validates, and materializes these declarations using standardized system primitives built directly into the host platform.

2.1.2 1.2 The "No Arbitrary Executable Code in Modules" Guarantee

Under UNIVA rules, **modules never ship unvetted executable logic in the backend process**.

- **Database Schema:** Created via fixed DDL generators driven by `models/*.json`.
- **Security Policies:** Seeded into system RBAC/ABAC tables driven by `security/access.json`.
- **Cross-Module Behavior:** Executed via named, versioned integration function contracts (`integrations/uses.json` and `functions/exposes.json`) dispatched through the host's controlled `FunctionRegistry`.
- **UI Customization:** Rendered dynamically via standard layout renderers, or loaded as static CSS/JS bundles inside sandboxed `<iframe sandbox="allow-scripts">` containers using the browser `postMessage` protocol.

Security Policies & Role Ownership Boundary

A module's `security/access.json` should only ever seed access rules that reference existing permission points — which models, fields, and features a role may touch. Roles themselves — their creation, and which real user account holds which role — are owned entirely by the platform's user-management system (`app_user` , `role` , `user_role`). A module declares *what can be granted*; only the platform decides *who it's granted to*.

2.1.3 1.3 Zero-Preinstalled-Tables Engine Architecture

The host application starts up with **0 pre-installed domain tables**. System tables (`account` , `app user` , `module catalog` , `account_module_install` , `sys_model` , `sys_field` , `role` , `user_role` , `role_model_access`) form the core infrastructure platform.

When a tenant admin clicks "Install" on a module (e.g. `farm`s or `invoicing`):

- 1. The engine reads the declarative definitions from the module folder.
- 2. The Database Engine creates physical SQL tables (`mod_farms_farm` , `mod_invoicing_invoice`).
- 3. The Security Engine seeds tenant roles and access control rules.
- 4. The UI Engine registers view schemas into `sys_view` .
- 5. The sidebar menu updates dynamically without restarting the server or refreshing the frontend bundle.

2.1.4 1.4 Frontend Live-Rendering ("No Swap / No Rebuild") Paradigm

Installing or upgrading a module requires **zero frontend rebuilds or Webpack/Vite compiles**.

The frontend application contains a generic, dynamic rendering engine (`DynamicModulePage.jsx`). When a user accesses a module route, the frontend fetches view specifications from the installer service (`/api/modules/:module_name/views`). The renderer dynamically generates tables, action buttons, progress bars, status badges, and modal forms directly from metadata.

If a module includes custom UI components (`views/assets/widget.js`), the frontend loads those assets inside an isolated `<iframe>` element. The `iframe` communicates with the parent app strictly via structured JSON messages (`postMessage`), ensuring custom UI widgets cannot directly access local storage, authentication tokens, or global window variables.

2.1.5 1.5 System Component Blueprint

The system architecture is decoupled into distinct component layers:

COMPONENT LAYER	RESPONSIBILITY
UNIVA Installer Engine	5-Engine Installer, Data REST APIs, Function Registry, System Tables (<code>sys_model</code> / <code>sys_field</code>).
Dynamic UI Renderer	Dynamic Module Views (<code>DynamicModulePage.jsx</code>), Sandboxed IFrame Host (<code>postMessage</code> protocol).

2.1.6 1.6 Engine Implementation Maturity Matrix

To maintain transparency as the platform evolves, the following matrix tracks the implementation status of each engine:

ENGINE COMPONENT	IMPLEMENTATION STATUS	CURRENT CAPABILITIES
Engine 1 — Manifest Engine	Implemented	Manifest parsing, SemVer checks, prerequisite verification, <code>requires_integration</code> checks.
Engine 2 — Database Engine	Implemented	Safe DDL generation, <code>sys_model</code> / <code>sys_field</code> registration, partial unique indexes (<code>WHERE deleted_at IS NULL</code>).
Engine 3 — Integration Engine	Implemented	Phase 2 Foreign Keys (<code>sys_field_relation</code>), namespaced target resolution (<code>module.model</code>), <code>FunctionRegistry</code> dispatcher.
Engine 4 — Security Engine	Implemented	Role seeding, <code>role_model_access</code> CRUD matrices, <code>role_field_access</code> rules, ABAC row-level filters (<code>role_record_rule</code>).
Engine 5 — UI Engine	Implemented	<code>views/*.json</code> registration, fallback auto-generator, static asset manifest, sandboxed <code>IFrame</code> <code>postMessage</code> protocol.

2.1.7 1.7 SQL Identifier Validation & Injection Defense

To prevent SQL injection vulnerabilities when building DDL dynamically (`ALTER TABLE` , `ADD CONSTRAINT` , `CREATE INDEX`), the engine enforces strict regex validation on all SQL identifiers prior to DDL string interpolation.

```
import re

IDENTIFIER_RE = re.compile(r"^[a-z0-9_]{1,63}$")

class InvalidIdentifierError(Exception):
    pass

def assert_safe_identifier(value: str, kind: str = "identifier") -> str:
    """Raise InvalidIdentifierError if value is not a safe SQL identifier. Returns value
    unchanged if safe."""
    if not isinstance(value, str) or not IDENTIFIER_RE.match(value):
        raise InvalidIdentifierError(
            f"Invalid {kind} '{value}': must match regex ^[a-z0-9_]{1,63}$"
        )
    return value
```

Every database operation splices identifiers ONLY after passing through `assert_safe_identifier()` .

2.1.8 1.8 Where Does Business Logic Live in UNIVA Modules?

Because UNIVA modules **never ship arbitrary executable server code directly inside the module package**, business logic is split cleanly across **3 controlled channels**:

```

                                flowchart TD
                                Root(["<i class='fa-solid fa-brain'></i> WHERE DOES BUSINESS LOGIC LIVE?"])

                                Root --> C1(["<i class='fa-solid fa-network-wired'></i> 1. Server Integration Functions<br/><code>functions/exposes.json</code> & <code>uses.json</code><br/>Host-mediated contracts"])
                                Root --> C2(["<i class='fa-solid fa-shield-halved'></i> 2. Sandboxed Lifecycle Hooks<br/><code>hooks/hooks.json</code><br/>5.0s / 128MB isolated execution"])
                                Root --> C3(["<i class='fa-solid fa-window-maximize'></i> 3. Client IFrame Widgets<br/><code>views/assets/</code><br/>Sandboxed postMessage protocol"])

                                style Root fill:#0f172a,stroke:#38bdf8,stroke-width:2px,color:#ffffff
                                style C1 fill:#eff6ff,stroke:#3b82f6,stroke-width:2px,color:#1e3a8a
                                style C2 fill:#fffbeb,stroke:#f59e0b,stroke-width:2px,color:#78350f
                                style C3 fill:#f5f3ff,stroke:#8b5cf6,stroke-width:2px,color:#4c1d95

```

1. **Host Integration Functions** (`functions/exposes.json` & `integrations/uses.json`):
 2. For host-mediated calculations, data transformations, or cross-module logic (e.g. `invoicing.get_invoice_total`).
 3. Defined as schema-validated JSON contracts in `exposes.json` .
 4. Implementation handlers are registered in the host platform's `FunctionRegistry` and called safely with schema validation.
5. **Event Lifecycle Hooks** (`hooks/hooks.json`):
 6. For installation, upgrade, or uninstallation events (e.g. seeding default data).
 7. Executed inside isolated process sandboxes with resource limits (5.0s timeout, 128MB memory limit).
8. **Interactive Client View Logic** (`views/assets/widget.js`):
 9. For complex UI interactions, custom charts, or multi-step form wizards.
10. Executed inside sandboxed `<iframe sandbox="allow-scripts">` containers communicating via `postMessage` .

2.1.9 Command Line Code Reference

```

# Command line to run tests verifying identifier validation regex
python -c "from assert_safe import assert_safe_identifier;
print(assert_safe_identifier('valid_name_123'))"

```

2.2.2. The Two Fundamental Module Types

In the UNIVA Module System, every module is strictly classified as either a **Main Module** or an **Extension Module**.

```

                                flowchart TD
                                Root(["<i class='fa-solid fa-cubes'></i> UNIVA MODULE CLASSIFICATION"])

                                Root --> Main(["<i class='fa-solid fa-building-columns'></i> Main Module<br/><code>module_type: 'main'</code>"])
                                Root --> Ext(["<i class='fa-solid fa-puzzle-piece'></i> Extension Module<br/><code>module_type: 'extension'</code>"])

                                Main --> M1(["<i class='fa-solid fa-database'></i> Owns models & SQL tables"])
                                Main --> M2(["<i class='fa-solid fa-table-cells'></i> Owns UI list/form views"])
                                Main --> M3(["<i class='fa-solid fa-user-shield'></i> Owns security RBAC/ABAC"])
                                Main --> M4(["<i class='fa-solid fa-bolt'></i> Hosts wizards & hooks"])

                                Ext --> E1(["<i class='fa-solid fa-ban'></i> Zero physical database tables"])
                                Ext --> E2(["<i class='fa-solid fa-desktop'></i> No standalone sidebar links"])
                                Ext --> E3(["<i class='fa-solid fa-square-plus'></i> Injects buttons & export tabs"])
                                Ext --> E4(["<i class='fa-solid fa-sliders'></i> Live metadata-driven extenders"])

                                style Root fill:#0f172a,stroke:#3b82f6,stroke-width:2px,color:#ffffff
                                style Main fill:#1e3a8a,stroke:#3b82f6,stroke-width:2px,color:#ffffff
                                style Ext fill:#065f46,stroke:#10b981,stroke-width:2px,color:#ffffff
                                style M1 fill:#eff6ff,stroke:#60a5fa,color:#1e3a8a
                                style M2 fill:#eff6ff,stroke:#60a5fa,color:#1e3a8a
                                style M3 fill:#eff6ff,stroke:#60a5fa,color:#1e3a8a
                                style M4 fill:#eff6ff,stroke:#60a5fa,color:#1e3a8a
                                style E1 fill:#ecfdf5,stroke:#34d399,color:#065f46
                                style E2 fill:#ecfdf5,stroke:#34d399,color:#065f46
                                style E3 fill:#ecfdf5,stroke:#34d399,color:#065f46
                                style E4 fill:#ecfdf5,stroke:#34d399,color:#065f46

```

2.2.1 2.1 Main Modules (`module_type: "main"`)

A **Main Module** represents a complete, self-contained business domain.

- **Data Layer:** Defines physical database tables in `models/*.json` (e.g. `mod_farms_farm` , `mod_farms_sensor`).
- **UI Layer:** Defines primary list and form views in `views/*.json` (e.g. `farm_list.json` , `farm_form.json`).
- **Security Layer:** Defines default roles and access policies in `security/access.json` .
- **Navigation:** Appears as a top-level sidebar navigation item.
- **Examples:** `farms` , `invoicing` , `inventory` , `schedule` , `support` , `alerts` , `reports` , `users` .

2.2.2 2.2 Extension Modules (module_type: "extension")

An **Extension Module** enhances or augments existing Main Modules without owning standalone database tables.

- **Data Layer:** Does NOT create database tables (`models: []`).
- **UI Layer:** Injects buttons, actions, tab panels, or export tools into existing Main Module views via `ui_extensions[]` .
- **Functionality:** Reuses existing APIs or provides specialized client-side transformations (e.g. PDF generation, CSV exports, specialized calculations).
- **Navigation:** Does NOT create separate sidebar navigation items; instead, injects actions into host views.
- **Examples:** `pdf_exporter` .

2.2.3 2.3 Feature Matrix & Feature Comparison

FEATURE CAPABILITY	MAIN MODULE ("MAIN")	EXTENSION MODULE ("EXTENSION")
Manifest <code>module_type</code>	"main"	"extension"
Database Models (<code>models/</code>)	Allowed (<code>models: ["invoice", ...]</code>)	Prohibited (<code>models: []</code>)
Physical Tables Created	Creates <code>mod_<module>_<model></code>	Zero physical SQL tables created
Sidebar Navigation	Renders primary sidebar link	No sidebar link created
UI View Definitions (<code>views/</code>)	Declares full list & form views	Injects UI extension widgets
Custom Widget Assets	Supported in <code>views/assets/</code>	Supported in <code>views/assets/</code>
Integration Function Exposing	Supported (<code>functions/exposes.json</code>)	Supported (<code>functions/exposes.json</code>)
Integration Function Calling	Supported (<code>integrations/uses.json</code>)	Supported (<code>integrations/uses.json</code>)
RBAC Role Seeding	Seeds default domain roles	Extends existing role permissions

2.2.4 Command Line Validation Code

```
# Command line to check module types across manifests using PowerShell
Get-ChildItem -Path "modules" -Filter "manifest.json" -Recurse | ForEach-Object {
    $m = Get-Content $_.FullName | ConvertFrom-Json
    [PSCustomObject]@{ Module = $m.name; Type = $m.module_type; Version = $m.version }
}
```

2.3 3. The Five-Engine Installer Orchestration Architecture

2.3.1 3.1 Overview of the 5-Engine Pipeline

When `install_module(account_id, module_name, user_id)` is invoked, the installer coordinates five specialized sub-engines in strict order:

```

flowchart TD
    Core(["<i class='fa-solid fa-gears'></i> InstallerEngine.install(account_id, module_name)"])
    Core --> E1(["<i class='fa-solid fa-file-code'></i> Engine 1: Manifest Loader<br/>SemVer & prerequisite validation"])
    E1 --> E2(["<i class='fa-solid fa-database'></i> Engine 2: Database Engine<br/>DDL Creation & System Catalog"])
    E2 --> E3(["<i class='fa-solid fa-link'></i> Engine 3: Integration Engine<br/>Phase 2 Foreign Keys & Functions"])
    E3 --> E4(["<i class='fa-solid fa-user-lock'></i> Engine 4: Security Engine<br/>RBAC & ABAC Seeding"])
    E4 --> E5(["<i class='fa-solid fa-palette'></i> Engine 5: UI Engine<br/>View Schemas & Asset Manifests"])

    style Core fill:#0f172a,stroke:#38bdf8,stroke-width:3px,color:#ffffff
    style E1 fill:#eff6ff,stroke:#3b82f6,stroke-width:2px,color:#1e3a8a
    style E2 fill:#f0f9ff,stroke:#0284c7,stroke-width:2px,color:#0369a1
    style E3 fill:#f0fdf4,stroke:#16a34a,stroke-width:2px,color:#15803d
    style E4 fill:#fffbeb,stroke:#d97706,stroke-width:2px,color:#78350f
    style E5 fill:#faf5ff,stroke:#9333ea,stroke-width:2px,color:#6b21a8

```

- Engine 1 — Manifest Engine:** Loads `manifest.json`, validates identifiers using `assert_safe_identifier()`, checks module prerequisites (`depends`), and confirms integration function availability (`requires_integration`).
- Engine 2 — Database Engine:** Parses `models/*.json`, diffs specs against system catalog (`sys_model / sys_field`), constructs safe DDL statements, creates physical SQL tables with system columns (`id`, `account_id`, `created_at`, `updated_at`, `deleted_at`), creates partial unique indexes (`WHERE deleted_at IS NULL`), and creates relation fields as plain UUID columns first.
- Engine 3 — Integration Engine:** Phase 2 link resolution — verifies target models exist using namespaced target resolution (`"module name.model name"`) and applies Foreign Key constraints (`sys_field_relation`). Validates and registers exposed function contracts (`functions/exposes.json`).
- Engine 4 — Security Engine:** Reads `security/access.json` and `default data/roles.json`, creates tenant roles (`role`), seeds model access permissions (`role_model_access`), field access rules (`role_field_access`), and ABAC domain filters (`role_record_rule`).
- Engine 5 — UI Engine:** Reads `views/*.json` schemas, generates fallback list/form views for unconfigured models, registers views into `sys_view`, registers static custom widget bundles (`views/assets/`) into `sys_module_assets`, and invalidates UI caches.

2.3.2 3.2 Stage-by-Stage Execution Sequence (`InstallerEngine.install`)

The installer executes the following sequence for a single `install(account_id, module_name, installed_by_user_id)` call:

- **Stage 0 — Guard & Lock:** Account ID and module name are validated as safe identifiers. Check whether module is already installed. Acquire atomic lock `install:{account_id}:{module_name}` .
- **Stage 1 — Manifest Validation (Engine 1):** `manifest.json` loaded and validated against JSON schema. Dependencies in `depends` and `requires_integration` checked.
- **Stage 2 — Schema Materialization (Engine 2):** Models diffed against `sys_model` / `sys_field` . Physical SQL tables created with 5 system columns. Relation fields created as UUID columns. Partial unique indexes created (`WHERE deleted_at IS NULL`).
- **Stage 3 — Link & Function Resolution (Engine 3):** Relation fields resolved to target models (`"module.model"`). Foreign keys attached via `ALTER TABLE` . exposed function contracts registered.
- **Stage 4 — Security Seeding (Engine 4):** Roles and permission rules seeded into `role_model_access` , `role_field_access` , `role_record_rule` .
- **Stage 5 — UI Registration (Engine 5):** View specs registered into `sys_view` ; widget bundles registered into `sys_module_assets` .
- **Stage 6 — Lifecycle Hook (`on_install`):** Module installation hook runs in isolated sandbox process.
- **Stage 7 — Record Install State:** Entry written to `sys_installed_module` .
- **Stage 8 — Commit:** Database transaction commits atomically.
- **Stage 9 — Cache Invalidation:** Account permission and view caches cleared.

2.3.3 3.3 Transaction Management, Rollbacks, and Concurrency Locking

To prevent partial installations, corrupted schemas, or race conditions during simultaneous module installations across multi-tenant accounts:

- **Concurrency Locking:** An atomic lock (`install:{account_id}:{module_name}`) is acquired before installation begins.
- **Database Transaction Scope:** All DDL executions, `sys_model` / `sys_field` registrations, role seedings, and view registrations execute inside a single transactional block.
- **Automated Rollback:** Any failure during schema creation, dependency checking, FK constraint addition, or hook execution triggers an immediate database transaction rollback (`ROLLBACK`).
- **Cache Invalidation:** Permission caches and model registries for the target account are invalidated ONLY after a successful transaction commit.

2.3.4 Command Line Execution Code

```
# Command line to trigger module install via API using PowerShell
$body = @{"account_id" = "acc_1001"; module_name = "farms" } | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8001/api/modules/farms/install" -Method Post -Body $body -ContentType "application/json"
```

2.4 4. Complete Declarative Module Folder Structure

2.4.1 4.1 Master Directory Layout Diagram

modules/<module_name>/	
├─ manifest.json	← REQUIRED: Manifest specification (Engine 1)
├─ models/	← REQUIRED: Declarative JSON model specifications (Engine 2)
│ └─ <model_1>.json	← Declarative field specs & relationships
│ └─ <model_2>.json	
├─ security/	← REQUIRED: RBAC & ABAC Security Policy Declarations (Engine 4)
│ └─ access.json	← Role definitions, model access, field access rules
├─ views/	← REQUIRED: UI View Specifications (Engine 5)
│ └─ <model_1>_list.json	← Data Table column specs & widget options
│ └─ <model_1>_form.json	← Form layout, section definitions & field inputs
│ └─ assets/	← OPTIONAL: Custom UI widget assets (IFrame sandboxed)
│ │ └─ style.css	← Custom styling for widget
│ │ └─ widget.js	← Sandboxed JavaScript (postMessage protocol)
├─ integrations/	← OPTIONAL: Cross-module required function specs (Engine 3)
│ └─ uses.json	← Declares dependencies on other modules' exposed functions
├─ functions/	← OPTIONAL: Cross-module exposed function contracts (Engine 3)
│ └─ exposes.json	← Declares integration functions offered to other modules
├─ default_data/	← OPTIONAL: Default seed data
│ └─ roles.json	← Additional system roles
│ └─ field_rules.json	← Default field-level validation rules
├─ migrations/	← OPTIONAL: Versioned schema migration files
│ └─ 1.0.0_to_1.1.0.json	← Incremental field additions & alterations
└─ hooks/	← OPTIONAL: Sandboxed lifecycle event hooks
└─ hooks.json	← Configures on_install and on_uninstall callbacks

2.4.2 4.2 Detailed Inventory of File Responsibilities

FILE PATH	FORMAT	REQUIRED	ENGINE RESPONSIBLE	PRIMARY PURPOSE
manifest.json	JSON	YES	Engine 1 (Manifest)	Module metadata, version, dependencies, model list, function contracts.
models/ <model>.json	JSON	YES (for Main)	Engine 2 (Database)	Field names, data types, validation constraints, relationship targets.
security/ access.json	JSON	YES	Engine 4 (Security)	Role-to-model access matrix, field-level read/write rules, ABAC domain filters.
views/ <model>_list.json	JSON	Recommended	Engine 5 (UI)	List view grid columns, sorting, filtering, action buttons, column widgets.
views/ <model>_form.json	JSON	Recommended	Engine 5 (UI)	Form layout sections, field input controls, widget options, validation rules.
views/assets/*	CSS/JS	Optional	Engine 5 (UI)	Custom widget assets loaded exclusively in sandboxed <code><iframe sandbox="allow-scripts"></code> .
integrations/ uses.json	JSON	Optional	Engine 3 (Integration)	Declarative list of integration functions required from other installed modules.
functions/ exposes.json	JSON	Optional	Engine 3 (Integration)	Exposed integration function definitions with strict JSON input/output schemas.
migrations/ *.json	JSON	Optional	Engine 2 (Database)	Incremental schema evolution DDL instructions for module upgrades.
hooks/ hooks.json	JSON	Optional	Lifecycle Engine	Declarative bindings for sandboxed <code>on_install</code> and <code>on_uninstall</code> events.

2.4.3 4.3 Mandatory vs Optional Files Checklist

- [x] manifest.json (Mandatory for all modules)
- [x] models/*.json (Mandatory for Main Modules; prohibited for Extension Modules)
- [x] security/access.json (Mandatory for all modules)
- [] views/*.json (Optional; if missing, Engine 5 auto-generates view specs from `sys_field`)
- [] views/assets/ (Optional; required only if module uses custom IFrame widgets)
- [] integrations/uses.json (Optional; required only if module calls another module's exposed functions)
- [] functions/exposes.json (Optional; required only if module offers functions to other modules)

- `migrations/*.json` (Optional; required only when upgrading module versions)
-

2.4.4 Command Line Directory Audit Code

```
# Command line to inspect folder tree for a specific module
Get-ChildItem -Path "modules/invoicing" -Recurse | Select-Object RelativeName, Length
```


3. Five-Engine Specifications

3.1 5. Engine 1 — Manifest Engine & Specification (`manifest.json`)

3.1.1 5.1 Complete JSON Schema & Validation Rules

`manifest.json` MUST be valid JSON and conform strictly to the following draft-07 specification:

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "UNIVA Module Manifest",
  "type": "object",
  "required": ["name", "version", "module_type", "models"],
  "properties": {
    "name": {
      "type": "string",
      "pattern": "^[a-z0-9_]{1,63}$",
      "description": "Unique identifier of the module (lowercase letters, numbers,
underscores).",
    },
    "display_name": {
      "type": "string",
      "description": "Human-readable module title displayed in Module Store.",
    },
    "version": {
      "type": "string",
      "pattern": "^[0-9]+\\.[0-9]+\\.[0-9]+$",
      "description": "Semantic versioning string (e.g. 1.0.0).",
    },
    "module_type": {
      "type": "string",
      "enum": ["main", "extension"],
      "description": "Module type classification.",
    },
    "target_app": {
      "type": "string",
      "default": "all",
      "description": "Target frontend application context ('sis_web', 'all').",
    },
    "depends": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Prerequisite module names that MUST be installed prior to this module.",
    },
    "models": {
      "type": "array",
      "items": { "type": "string", "pattern": "^[a-z0-9_]{1,63}$" },
      "description": "List of model names owned by this module.",
    },
    "requires_integration": {
      "type": "array",
      "items": { "type": "string" },
      "description": "List of required integration functions in 'module_name.function_name'
format.",
    },
    "exposes_integration": {
      "type": "array",
      "items": { "type": "string" },
      "description": "List of integration functions exposed by this module.",
    },
    "default_roles": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Role identifiers seeded into system upon installation.",
    },
  },
}

```

```

    "description": {
      "type": "string",
      "description": "Detailed description of module capabilities."
    }
  }
}

```

3.1.2 5.2 Extended Dependency Declarations (depends , requires_integration , exposes_integration)

```

{
  "name": "invoicing",
  "display_name": "Invoicing & Billing Management",
  "version": "1.0.0",
  "module_type": "main",
  "target_app": "sis_web",
  "depends": ["contacts_base"],
  "models": ["invoice", "invoice_line"],
  "default_roles": ["invoicing_manager", "billing_viewer"],
  "requires_integration": [
    "contacts_base.get_contact",
    "users.get_user"
  ],
  "exposes_integration": [
    "invoicing.get_invoice_total",
    "invoicing.list_open_invoices"
  ],
  "description": "Enterprise billing, invoice lines, payment tracking, and financial integration."
}

```

1. **depends** **Check:** Before installing `invoicing` , Engine 1 verifies that `contacts_base` is already installed and enabled for `account_id` . If missing, install halts immediately with `ModuleInstallError` .
2. **requires_integration** **Check:** Engine 1 verifies that `contacts_base.get_contact` and `users.get_user` function contracts are registered and exposed in the system function catalog.

3.1.3 5.3 Manifest Reference Table

FIELD	REQUIRED	TYPE	VALIDATION CONSTRAINT	DESCRIPTION
name	YES	string	Regex <code>^[a-z0-9_]{1,63}\$</code>	Unique module system identifier.
display_name	No	string	Max 150 chars	Human-readable title.
version	YES	string	SemVer <code>X.Y.Z</code>	Version indicator.
module_type	YES	string	"main" or "extension"	Module classification.
target_app	No	string	"sis_web" or "all"	Target application environment.
depends	No	array	List of module names	Hard prerequisite module list.
models	YES	array	List of safe identifiers	Models owned by this module.
requires_integration	No	array	List of "module.func"	Machine-checkable function dependencies.
exposes_integration	No	array	List of "module.func"	Exposed function contracts provided to system.
default_roles	No	array	List of role name strings	Default role names seeded at install time.

3.1.4 Command Line Validation Code

```
# Command line to validate all manifest files against JSON syntax
Get-ChildItem -Path "modules" -Filter "manifest.json" -Recurse | ForEach-Object {
    try {
        $json = Get-Content $_.FullName | ConvertFrom-Json
        Write-Host "Valid manifest: $($json.name) v$($json.version)" -ForegroundColor Green
    } catch {
        Write-Host "Invalid manifest JSON in $($_.FullName)" -ForegroundColor Red
    }
}
```

3.2 6. Engine 2 — Database Engine & Declarative Schemas

(models/*.json)

3.2.1 6.1 Database Engine Responsibilities & Materialization Workflow

The **Database Engine** is the sole authority responsible for physical SQL DDL execution. It transforms declarative JSON model specifications (models/*.json) into physical database tables.

```

                                flowchart TD
    Start(["<i class='fa-solid fa-file-invoice'></i> Read models/model.json"]) --> Check{"<i class='fa-solid fa-magnifying-glass'></i> Check sys_model catalog"}

    Check -->|Table missing| Create(["<i class='fa-solid fa-table'></i> Execute CREATE  
TABLE<br/>with 5 Mandatory System Columns"])
    Check -->|Table exists| Diff(["<i class='fa-solid fa-code-compare'></i> Diff fields against  
sys_field<br/>& execute ALTER TABLE"])

    Create --> Reg(["<i class='fa-solid fa-folder-tree'></i> Register sys_model & sys_field"])
    Diff --> Reg

    style Start fill:#1e3a8a,stroke:#3b82f6,color:#ffffff
    style Check fill:#0f172a,stroke:#38bdf8,color:#ffffff
    style Create fill:#ecfdf5,stroke:#10b981,color:#065f46
    style Diff fill:#fffbeb,stroke:#f59e0b,color:#78350f
    style Reg fill:#f5f3ff,stroke:#8b5cf6,color:#4c1d95
  
```

3.2.2 6.2 Supported Data Types & SQL Type Mapping Table

Engine 2 translates high-level model types into standard SQL column types:

DECLARATIVE TYPE	POSTGRESQL DDL TYPE	SQLITE DDL TYPE	TYPESCRIPT UI TYPE	DEFAULT COLUMN CONSTRAINT
"string"	TEXT	TEXT	string	NULL
"numeric"	NUMERIC (15,2)	REAL	number	NULL
"integer"	INTEGER	INTEGER	number	NULL
"float"	DOUBLE PRECISION	REAL	number	NULL
"boolean"	BOOLEAN	INTEGER	boolean	DEFAULT FALSE
"date"	DATE	TEXT	string (YYYY-MM-DD)	NULL
"datetime"	TIMESTAMP	TEXT	string (ISO-8601)	NULL
"uuid"	UUID	TEXT	string (UUIDv4)	NULL
"json"	JSONB	TEXT	object / array	NULL
"relation"	UUID	TEXT	string (UUIDv4)	Plain UUID first (Phase 2 FK)

3.2.3 6.3 Mandatory System Columns (id , account_id , created_at , updated_at , deleted_at)

Every physical SQL table created by Engine 2 MUST contain the following 5 system columns:

```
CREATE TABLE mod_invoicing_invoice (
  id VARCHAR(36) PRIMARY KEY,
  account_id VARCHAR(36) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  deleted_at TIMESTAMP NULL,

  -- Declarative Model Fields
  number TEXT NOT NULL,
  customer_id VARCHAR(36) NULL,
  total_amount NUMERIC(15,2) NULL,
  status TEXT DEFAULT 'draft'
);
```

- **id** : Unique primary key UUID (UUIDv4 string).
- **account_id** : Multi-tenant account isolation key. Every query executed via `chokepoint.py` automatically filters by `account_id`.
- **created_at** : Timestamp when record was created.
- **updated_at** : Timestamp when record was last updated.
- **deleted_at** : Soft-delete timestamp (`NULL` indicates active record). Physical `DELETE` SQL queries are strictly prohibited; records are soft-deleted by setting `deleted_at = NOW()`.

3.2.4 6.4 Soft Delete vs Unique Constraints (Partial Unique Indexes)

Problem Solved: Records are soft-deleted (`deleted_at = NOW()`). Standard unique constraints (`UNIQUE(number)`) would prevent reusing a natural key (e.g. `invoice.number = "INV-1001"`) if an older deleted row exists.

Solution: Whenever a field spec declares `"unique": true`, Engine 2 constructs a **Partial Unique Index** scoped strictly to active rows (`WHERE deleted_at IS NULL`).

PostgreSQL & SQLite 3.8+ DDL:

```
CREATE UNIQUE INDEX ux_mod_invoicing_invoice_number
ON mod_invoicing_invoice (account_id, number)
WHERE deleted_at IS NULL;
```

This guarantees uniqueness across active records while allowing natural key reuse after soft deletion.

3.2.5 6.5 Model JSON Schema Specifications & Field Attributes

Example: `modules/invoicing/models/invoice.json` :

```
{
  "name": "invoice",
  "table_name": "mod_invoicing_invoice",
  "label": "Customer Invoice",
  "description": "Customer billing invoice header record",
  "fields": [
    {
      "name": "number",
      "type": "string",
      "required": true,
      "unique": true,
      "label": "Invoice Number",
      "description": "Unique billing invoice reference number",
      "searchable": true
    },
    {
      "name": "customer_id",
      "type": "relation",
      "relation_target": "contacts_base.contact",
      "required": true,
      "label": "Customer Contact",
      "description": "Foreign key reference to contacts_base contact model"
    },
    {
      "name": "total_amount",
      "type": "numeric",
      "required": true,
      "default": 0.00,
      "label": "Total Invoice Amount"
    },
    {
      "name": "status",
      "type": "string",
      "required": true,
      "default": "draft",
      "options": ["draft", "sent", "paid", "cancelled"],
      "label": "Invoice Status"
    }
  ]
}
```

3.2.6 6.6 System Tables (sys_model , sys_field) Schema

Engine 2 maintains the system catalog in sys_model and sys_field :

```
CREATE TABLE sys_model (  
  id VARCHAR(36) PRIMARY KEY,  
  account_id VARCHAR(36) NOT NULL,  
  module_name VARCHAR(100) NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  table_name VARCHAR(120) NOT NULL,  
  label VARCHAR(150),  
  description VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(account_id, module_name, name)  
);  
  
CREATE TABLE sys_field (  
  id VARCHAR(36) PRIMARY KEY,  
  model_id VARCHAR(36) REFERENCES sys_model(id) ON DELETE CASCADE,  
  name VARCHAR(100) NOT NULL,  
  field_type VARCHAR(50) NOT NULL,  
  required BOOLEAN DEFAULT FALSE,  
  default_value VARCHAR(255),  
  options JSONB,  
  is_searchable BOOLEAN DEFAULT TRUE,  
  relation_target VARCHAR(100),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(model_id, name)  
);
```

3.2.7 6.7 Database Engine Materialization Logic

```

import os
import json
import uuid
from typing import List, Any
from sqlalchemy import Column, String, Integer, Float, Boolean, DateTime, Table, MetaData, inspect, text

class DatabaseEngine:
    TYPE_MAP = {
        "string": String(500),
        "numeric": Float,
        "integer": Integer,
        "float": Float,
        "boolean": Boolean,
        "date": DateTime,
        "datetime": DateTime,
        "uuid": String(36),
        "json": String(2000),
        "relation": String(36) # Relation columns created as String(36) UUID first
    }

    def __init__(self, engine, db_session):
        self.engine = engine
        self.db = db_session

    def materialize_schema(self, module, account_id: str) -> List[Any]:
        safe_account_id = assert_safe_identifier(account_id, "account ID")
        created_sys_models = []
        inspector = inspect(self.engine)
        existing_tables = inspector.get_table_names()

        for model_name in module.models:
            safe_model_name = assert_safe_identifier(model_name, "model name")
            model_file = os.path.join(module.directory, "models", f"{safe_model_name}.json")
            if not os.path.exists(model_file):
                continue

            with open(model_file, "r", encoding="utf-8") as f:
                model_spec = json.load(f)

            table_name = assert_safe_identifier(
                model_spec.get("table_name", f"mod_{module.name}_{safe_model_name}"), "table
name"
            )
            fields = model_spec.get("fields", [])

            # Phase 1: Physical SQL Table Creation
            if table_name not in existing_tables:
                metadata = MetaData()
                columns = [
                    Column("id", String(36), primary_key=True, default=lambda: str(uuid.uuid4())
                ),
                    Column("account_id", String(36), nullable=False),
                    Column("created_at", DateTime),
                    Column("updated_at", DateTime),
                    Column("deleted_at", DateTime, nullable=True)
                ]

```

```

        for field in fields:
            fname = assert_safe_identifier(field["name"], "field name")
            ftype = field.get("type", "string")
            sa_type = self.TYPE_MAP.get(ftype, String(500))
            nullable = not field.get("required", False)
            columns.append(Column(fname, sa_type, nullable=nullable))

        Table(table_name, metadata, *columns, extend_existing=True)
        metadata.create_all(bind=self.engine)
        metadata.clear()

        # Phase 1b: Partial Unique Indexes for unique fields
        for field in fields:
            if field.get("unique", False):
                fname = assert_safe_identifier(field["name"], "field name")
                idx_name = assert_safe_identifier(f"ux_{table_name}_{fname}", "index
name")

                idx_sql = f"""
                    CREATE UNIQUE INDEX {idx_name}
                    ON {table_name} (account_id, {fname})
                    WHERE deleted_at IS NULL;
                """
                try:
                    self.db.execute(text(idx_sql))
                except Exception as e:
                    logger.warning(f"Index creation notice on {idx_name}: {e}")

        # Phase 2: System Catalog Registration (sys_model & sys_field)
        sys_model = self._upsert_sys_model(safe_account_id, module.name, safe_model_name, t
able_name, model_spec)
        self._upsert_sys_fields(sys_model.id, fields)
        created_sys_models.append(sys_model)

    return created_sys_models

```

3.2.8 Command Line Schema Test Code

```

# Command line to inspect created table structures in SQLite
sqlite3 sis.db ".schema mod_invoicing_invoice"

```

3.3 7. Engine 3 — Integration Engine (Data Linking & Function Contracts)

3.3.1 7.1 Cross-Module Communication Design Principles

In multi-tenant architecture, modules MUST be decoupled. Module A (`invoicing`) cannot directly import Python modules or issue raw cross-table SQL joins against Module B (`contacts_base`).

The **Integration Engine** provides two disciplined communication channels:

1. **Namespaced Data Linking:** Declarative relation fields using `"module_name.model_name"` format that resolve into physical Foreign Key constraints once target models exist.
2. **Function Calling:** Contract-based, schema-validated behavior invocation dispatched via `FunctionRegistry` .

3.3.2 7.2 Mechanism A — Namespaced Data Linking (`"module.model"`) & Two-Phase FK Constraints

To prevent model name collisions across modules, `relation_target` MUST be fully qualified as `"target_module_name.target_model_name"` (e.g. `"contacts_base.contact"`).

- **Phase 1 (Engine 2):** `customer_id` is created as a plain `VARCHAR(36)` UUID column during initial DDL materialization.
- **Phase 2 (Engine 3):** Engine 3 parses `relation_target` , validates target module and model, verifies target table in `sys_model` , attaches Foreign Key constraint, and records dependency in `sys_field_relation` .

The resolution flow, step by step, for every relation field belonging to the module being installed:

```

flowchart TD
    Start(["<i class='fa-solid fa-rotate'></i> Loop: For each relation field"]) --> Q1(["<i class='fa-solid fa-circle-question'></i> Is relation_target in<br>'module.model' format?"])
    Q1 -->|NO| Stop1(["<i class='fa-solid fa-circle-xmark'></i> STOP & Rollback<br>Malformed target"])
    Q1 -->|YES| Q2(["<i class='fa-solid fa-circle-question'></i> Is target model installed<br>>in sys_model?"])
    Q2 -->|NO| Stop2(["<i class='fa-solid fa-circle-xmark'></i> STOP & Rollback<br>Target not installed"])
    Q2 -->|YES| Resolve(["<i class='fa-solid fa-shield-halved'></i> Validate SQL identifiers<br>& resolve table names"])
    Resolve --> Alter(["<i class='fa-solid fa-key'></i> Execute ALTER TABLE<br>ADD CONSTRAINT FOREIGN KEY"])
    Alter -->|SUCCESS| Rec(["<i class='fa-solid fa-circle-check'></i> Record link in sys_field_relation"])
    Alter -->|FAILURE| Fail(["<i class='fa-solid fa-triangle-exclamation'></i> Hard Failure & Rollback"])

    style Start fill:#0f172a,stroke:#38bdf8,color:#ffffff
    style Q1 fill:#1e3a8a,stroke:#3b82f6,color:#ffffff
    style Q2 fill:#1e3a8a,stroke:#3b82f6,color:#ffffff
    style Stop1 fill:#fef2f2,stroke:#ef4444,color:#991b1b
    style Stop2 fill:#fef2f2,stroke:#ef4444,color:#991b1b
    style Resolve fill:#f0f9ff,stroke:#0284c7,color:#0369a1
    style Alter fill:#ffffbe,stroke:#f59e0b,color:#78350f
    style Rec fill:#ecfdf5,stroke:#10b981,color:#065f46
    style Fail fill:#fef2f2,stroke:#ef4444,color:#991b1b

```

Resolution Error Enforcement

If the `ALTER TABLE ... ADD CONSTRAINT` statement fails, Engine 3 MUST treat it as a hard failure and trigger a transaction rollback (`ROLLBACK`), preventing orphan relation columns.

3.3.3 7.3 Foreign Key Tracking Table (`sys_field_relation`) & Safe Uninstall Checks

```

CREATE TABLE sys_field_relation (
  id VARCHAR(36) PRIMARY KEY,
  source_field_id VARCHAR(36) REFERENCES sys_field(id) ON DELETE CASCADE,
  target_model_id VARCHAR(36) REFERENCES sys_model(id) ON DELETE CASCADE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

When a user attempts to uninstall a module (e.g. `contacts_base`):

1. Engine 3 queries `sys_field_relation` for any active relations pointing to `contacts_base` models.
2. If `invoicing` has active relations pointing to `contacts_base`, the uninstallation is blocked with a clear warning: `"Cannot uninstall contacts_base: module 'invoicing' depends on field relation invoice.customer_id."`

3.3.4 7.4 Mechanism B — Function Calling (`integrations/uses.json` & `functions/exposes.json`)

When Module B needs to call logic owned by Module A:

`contacts_base/functions/exposes.json` :

```
{
  "functions": [
    {
      "name": "get_contact",
      "description": "Fetches formatted contact record by ID",
      "input_schema": {
        "contact_id": "string"
      },
      "output_schema": {
        "id": "string",
        "display_name": "string",
        "email": "string",
        "phone": "string"
      },
      "implementation": "contacts_base.services.get_contact"
    }
  ]
}
```

`invoicing/integrations/uses.json` :

```
{
  "requires": [
    {
      "module": "contacts_base",
      "function": "get_contact"
    }
  ]
}
```

3.3.5 7.5 Function Registry Runtime Dispatcher & Schema Validation Implementation

```

from typing import Dict, Any

class SchemaValidationError(Exception):
    pass

class FunctionRegistry:
    def __init__(self, db_session):
        self.db = db_session
        self.function_table = {}

    def register_implementation(self, function_key: str, handler_callable):
        self.function_table[function_key] = handler_callable

    def call(self, caller_module: str, target_module: str, function_name: str, args: Dict[str, Any], account_id: str) -> Dict[str, Any]:
        # 1. Verify caller declared dependency
        if not self._verify_declared_use(caller_module, target_module, function_name):
            raise PermissionError(f"Module '{caller_module}' did not declare dependency on '{target_module}.{function_name}' in integrations/uses.json")

        # 2. Verify target module is installed for account
        if not self._is_module_installed(account_id, target_module):
            raise ModuleInstallError(f"Target module '{target_module}' is not installed for account '{account_id}'")

        # 3. Fetch function schema definition
        func_spec = self._get_function_spec(target_module, function_name)
        self._validate_schema(args, func_spec["input_schema"])

        # 4. Dispatch invocation
        handler = self.function_table.get(func_spec["implementation"])
        if not handler:
            raise NotImplementedError(f"Implementation '{func_spec['implementation']}' is not registered.")

        result = handler(args, account_id)
        self._validate_schema(result, func_spec["output_schema"])
        return result

    def _validate_schema(self, data: Dict[str, Any], schema: Dict[str, str]):
        for key, expected_type in schema.items():
            if key not in data:
                raise SchemaValidationError(f"Missing required payload key: '{key}'")

```

3.3.6 Command Line Verification Code

```

# Command line to inspect active field relation links in database
sqlite3 sis.db "SELECT * FROM sys_field_relation;"

```

3.4 8. Engine 4 — Security Engine & RBAC/ABAC Declarations

(security/access.json)

3.4.1 8.1 Security Engine Responsibilities & Role Seeding

The **Security Engine** seeds access policies into the tenant's security tables:

1. `role` : Tenant-scoped role definitions (`name` , `can_install_modules`).
2. `role_model_access` : Model-level CRUD permissions (`can_create` , `can_read` , `can_write` , `can_delete`).
3. `role_field_access` : Field-level read/write permissions (`can_read` , `can_write`).
4. `role_record_rule` : Attribute-Based Access Control (ABAC) row-level filtering rules (`domain_filter`).

3.4.2 8.2 Declarative RBAC Matrix (security/access.json) Schema

```
{
  "roles": {
    "invoicing_manager": {
      "display_name": "Invoicing Manager",
      "can_install_modules": false,
      "model_access": {
        "invoice": { "create": true, "read": true, "update": true, "delete": true },
        "invoice_line": { "create": true, "read": true, "update": true, "delete": true }
      },
      "field_access": {
        "invoice": {
          "total_amount": { "read": true, "write": true },
          "status": { "read": true, "write": true }
        }
      }
    },
    "billing_viewer": {
      "display_name": "Billing Viewer",
      "can_install_modules": false,
      "model_access": {
        "invoice": { "create": false, "read": true, "update": false, "delete": false },
        "invoice_line": { "create": false, "read": true, "update": false, "delete": false }
      },
      "field_access": {
        "invoice": {
          "total_amount": { "read": true, "write": false }
        }
      }
    }
  }
}
```

3.4.3 8.3 ABAC Rules & Record-Level Domain Filters (domain_filter)

For fine-grained row-level authorization (e.g. "Farmers can only see farms in their assigned region"):

```
{
  "abac_rules": [
    {
      "role": "farmer",
      "model": "farm",
      "domain_filter": {
        "field": "region_id",
        "operator": "=",
        "value_source": "user_property.region_id"
      }
    }
  ]
}
```

When a user with `farmer` role queries `/api/data/farm`, the security engine inspects `role_record_rule` and automatically appends `AND region_id = :user_region_id` to the SQL query.

3.4.4 8.4 Security Engine Role Seeding Logic

```

import os
import json
import uuid
from typing import List, Any

class SecurityEngine:
    def __init__(self, db_session):
        self.db = db_session

    def seed_defaults(self, module, account_id: str, sys_models: List[Any]):
        sec_file = os.path.join(module.directory, "security", "access.json")
        if not os.path.exists(sec_file):
            return

        with open(sec_file, "r", encoding="utf-8") as f:
            sec_data = json.load(f)

        roles_dict = sec_data.get("roles", {})
        sys_model_map = {m.name: m for m in sys_models}

        for role_key, role_spec in roles_dict.items():
            role_row = self._upsert_role(account_id, role_key, role_spec)

            # Model access permissions
            model_access = role_spec.get("model_access", {})
            for model_name, perms in model_access.items():
                if model_name in sys_model_map:
                    sys_m = sys_model_map[model_name]
                    self._upsert_model_access(role_row.id, sys_m.id, perms)

    def _upsert_role(self, account_id, role_key, spec):
        from models.rbac import Role
        r = self.db.query(Role).filter(Role.account_id == account_id, Role.name == role_key).fi
rst()
        if not r:
            r = Role(
                id=str(uuid.uuid4()),
                account_id=account_id,
                name=role_key,
                can_install_modules=spec.get("can_install_modules", False)
            )
            self.db.add(r)
            self.db.flush()
        return r

    def _upsert_model_access(self, role_id, model_id, perms):
        from models.rbac import RoleModelAccess
        rma = self.db.query(RoleModelAccess).filter(
            RoleModelAccess.role_id == role_id,
            RoleModelAccess.model_id == model_id
        ).first()
        if not rma:
            rma = RoleModelAccess(
                role_id=role_id,
                model_id=model_id,
                can_create=perms.get("create", False),

```

```
        can_read=perms.get("read", False),  
        can_write=perms.get("update", False),  
        can_delete=perms.get("delete", False)  
    )  
    self.db.add(rma)
```

3.4.5 Command Line Security Matrix Inspection Code

```
# Command line to view security access JSON files across modules  
Get-ChildItem -Path "modules" -Filter "access.json" -Recurse | Select-Object FullName
```

3.5 9. Engine 5 — UI Engine & Live View Registration (`views/`)

3.5.1 9.1 View Spec Registration (`sys_view`) & Automatic Fallback Generator

The **UI Engine** registers view specifications into system tables (`sys_view` , `sys_module_assets`). If a module does not supply custom JSON view files, Engine 5 automatically inspects `sys_field` and auto-generates standard view specs.

```
import os
from typing import List, Any

class UIEngine:
    def __init__(self, db_session):
        self.db = db_session

    def register_views_and_assets(self, module, account_id: str, sys_models: List[Any]):
        for sys_m in sys_models:
            list_view_file = os.path.join(module.directory, "views", f"{sys_m.name}_list.json")
            form_view_file = os.path.join(module.directory, "views", f"{sys_m.name}_form.json")

            list_spec = self._read_or_generate_list(list_view_file, sys_m)
            form_spec = self._read_or_generate_form(form_view_file, sys_m)

            self._upsert_sys_view(account_id, sys_m.id, "list", list_spec)
            self._upsert_sys_view(account_id, sys_m.id, "form", form_spec)

        self._register_assets(module, account_id)

    def _generate_fallback_list(self, sys_m):
        fields = self.db.query(SysField).filter(SysField.model_id == sys_m.id).all()
        return {
            "view_id": f"view_{sys_m.name}_list",
            "model": sys_m.name,
            "type": "list",
            "title": f"{sys_m.label} Overview",
            "columns": [{"field": f.name, "label": f.name.replace("_", " ").title()} for f in f
            ields],
            "actions": ["create", "edit", "delete"]
        }
```

3.5.2 9.2 List View Schema (<model>_list.json) & Column Widgets

modules/invoicing/views/invoice_list.json :

```
{
  "view_id": "view_invoice_list",
  "model": "invoice",
  "type": "list",
  "title": "Invoices Overview",
  "columns": [
    { "field": "number", "label": "Invoice #", "sortable": true, "filterable": true },
    { "field": "customer_id", "label": "Customer", "widget": "relation",
"relation_label_field": "display_name" },
    { "field": "total_amount", "label": "Total Amount", "widget": "currency", "currency_code":
"USD" },
    { "field": "status", "label": "Status", "widget": "badge", "badge_colors": { "draft": "yell
ow", "paid": "green", "cancelled": "red" } }
  ],
  "actions": ["create", "edit", "delete"]
}
```

3.5.3 9.3 Form View Schema (<model>_form.json) & Section Layouts

modules/invoicing/views/invoice_form.json :

```
{
  "view_id": "view_invoice_form",
  "model": "invoice",
  "type": "form",
  "title": "Customer Invoice Form",
  "layout": {
    "sections": [
      {
        "title": "Header Information",
        "columns": 2,
        "fields": [
          { "name": "number", "widget": "input_text", "label": "Invoice Number", "required": true },
          { "name": "customer_id", "widget": "dropdown_select", "label": "Customer Contact", "required": true }
        ]
      },
      {
        "title": "Billing Summary",
        "columns": 2,
        "fields": [
          { "name": "total_amount", "widget": "input_currency", "label": "Total Amount" },
          { "name": "status", "widget": "select_dropdown", "label": "Status", "options": ["draft", "sent", "paid", "cancelled"] }
        ]
      }
    ]
  }
}
```

3.5.4 9.4 Asset Bundles (views/assets/) & Static Asset Manifests

If a module requires a custom widget, static files live in views/assets/ :

- views/assets/style.css
- views/assets/widget.js

Engine 5 registers these assets in sys_module_assets :

```
{
  "module": "invoicing",
  "version": "1.0.0",
  "assets": {
    "css": ["/module-assets/invoicing/1.0.0/style.css"],
    "js": ["/module-assets/invoicing/1.0.0/widget.js"]
  }
}
```

3.5.5 9.5 Sandboxed IFrame Architecture & `postMessage` Protocol Contract

Custom UI widgets are ONLY loaded inside a sandboxed `<iframe>` element. The iframe NEVER receives authentication tokens or direct database API access.

```
sequenceDiagram
    autonumber
    participant Host as 🌐 Parent Host Application
    participant Frame as 📄 Sandboxed IFrame (widget.js)

    Host->>Frame: postMessage({ type: "INIT_DATA", payload }, targetOrigin)
    Note over Frame: Renders custom widget, step wizard, or chart
    Frame->>Host: postMessage({ type: "SUBMIT", payload }, targetOrigin)
    Note over Host: Validates event.origin & executes host_data_write(payload)
```

views/assets/widget.js :

```
// Running strictly inside sandboxed iframe
window.addEventListener("message", (event) => {
    // Always validate message origin before processing payload
    if (event.origin !== window.location.origin) return;

    if (event.data && event.data.type === "INIT_DATA") {
        const record = event.data.payload;
        document.getElementById("title").innerText = record.number;
    }
});

function submitChanges(updatedPayload) {
    window.parent.postMessage({
        type: "SUBMIT",
        payload: updatedPayload
    }, window.location.origin);
}
```

3.5.6 Command Line UI View Spec Audit Code

```
# Command line to inspect registered views via REST endpoint
Invoke-RestMethod -Uri "http://localhost:8001/api/modules/invoicing/views" -Method Get
```

4. Operations & Mechanics

4.1 10. Lifecycle Event Hooks & Migrations (`hooks/` & `migrations/`)

4.1.1 10.1 Sandboxed Lifecycle Callbacks (`hooks/hooks.json`)

Lifecycle event hooks enable modules to execute custom setup logic when installed, upgraded, or uninstalled (e.g. seeding default master data or clearing cached resources).

To maintain platform stability, hooks do NOT run directly in the host server process. They run as sandboxed scripts declared in `hooks/hooks.json` :

`modules/farms/hooks/hooks.json` :

```
{
  "on_install": "hooks.on_install",
  "on_uninstall": "hooks.on_uninstall"
}
```

4.1.2 10.2 Hook Execution Sandbox Specifications

Lifecycle hooks execute under strict process isolation enforced by the `LifecycleEngine` prior to execution:

- **Process Isolation:** Each hook callback executes inside an isolated process sandbox with no direct access to host process memory, global singletons, or open database connection pools.
- **Restricted SDK Allowlist:** Hooks can only import and invoke functions exposed via `univa_hooks_sdk` (`get_record` , `emit_audit_log` , `call_integration`). Direct imports of host modules are forbidden.
- **Strict Resource Boundaries:** Hard timeout ceiling of **5.0 seconds** and memory ceiling of **128MB** per hook execution. Exceeding either limit forcefully terminates the process sandbox and rolls back the installation transaction.
- **Network Access Control:** Direct socket bindings and arbitrary HTTP network calls are blocked at container/process sandbox boundaries. Any cross-module network communication MUST route through declared `integrations/uses.json` function contracts.

4.1.3 10.3 Database Migration Engine (`migrations/<ver>_to_<ver>.json`) Schema & Execution

When upgrading an installed module to a newer version (e.g. `1.0.0` to `1.1.0`), the **Database Engine** executes declarative migration scripts (`migrations/1.0.0_to_1.1.0.json`):

`modules/invoicing/migrations/1.0.0_to_1.1.0.json :`

```
{
  "from_version": "1.0.0",
  "to_version": "1.1.0",
  "operations": [
    {
      "type": "add_column",
      "model": "invoice",
      "field": {
        "name": "notes",
        "type": "string",
        "required": false,
        "label": "Invoice Notes"
      }
    }
  ]
}
```

Engine 2 executes `ALTER TABLE mod_invoicing_invoice ADD COLUMN notes TEXT NULL;` and registers `notes` in `sys_field`.

4.1.4 Command Line Migration Check Code

```
# Command line to check migration JSON schema files
Get-ChildItem -Path "modules" -Filter "*.json" -Recurse | Where-Object { $_.DirectoryName -
like "**migrations*" }
```

4.2 11. System Architecture & Data Flow (Mermaid Diagrams)

4.2.1 11.1 Module Installation Overview Diagram

```

graph TD
    Client["🌐 Dynamic UI Renderer"]
    Installer["⚡ UNIVA Installer Engine"]

    subgraph Installer_Internal_Engines [Installer Internal Engines]
        E1["1 Manifest Engine"]
        E2["2 Database Engine"]
        E3["3 Integration Engine"]
        E4["4 Security Engine"]
        E5["5 UI Engine"]
        CK["🛡 Data REST API / Chokepoint"]
    end

    subgraph Storage [Storage]
        DB["📄 Database System"]
        ModDir["📁 modules/ Directory"]
    end

    Client -->|1. POST /api/modules/:name/install| Installer
    Installer --> E1
    E1 -->|Load & Validate| ModDir
    E1 --> E2
    E2 -->|DDL CREATE TABLE & sys_model| DB
    E2 --> E3
    E3 -->|Attach FK Constraints| DB
    E3 --> E4
    E4 -->|Seed Roles & RBAC| DB
    E4 --> E5
    E5 -->|Register Views & Assets| DB
    Client -->|3. GET /api/modules/:name/views| E5
    Client -->|4. CRUD /api/data/:model_name| CK
    CK --> DB
  
```

4.2.2 11.2 Module Installation Sequence Diagram

```
sequenceDiagram
    autonumber
    participant User as  Admin User
    participant Web as  sis_web (Port 5174)
    participant SIS as  sis_backend (Port 8001)
    participant E2 as  DatabaseEngine
    participant E3 as  IntegrationEngine
    participant DB as  SQLite (sis.db)

    User->>Web: Click "Install Module" (e.g. invoicing)
    Web->>SIS: POST /api/modules/invoicing/install
    SIS->>SIS: Verify JWT Token & Account Admin Role
    SIS->>E2: materialize_schema(invoicing)
    E2->>DB: CREATE TABLE mod_invoicing_invoice (...)
    E2->>DB: INSERT INTO sys_model & sys_field
    SIS->>E3: resolve_links(invoicing)
    E3->>DB: ALTER TABLE ... ADD CONSTRAINT fk_customer_id
    SIS->>DB: UPDATE account_module_install SET state='installed'
    SIS-->>Web: Return { success: true, message: "Installed successfully" }
    Web->>Web: Sidebar menu updates dynamically with "Invoicing"
```

4.2.3 11.3 Dynamic UI View Composition Sequence Diagram

```
sequenceDiagram
    autonumber
    participant Web as  sis_web (Port 5174)
    participant SIS as  sis_backend (Port 8001)
    participant DB as  SQLite (sis.db)

    Web->>SIS: GET /api/modules/invoicing/views
    SIS->>DB: Query sys_view for account_id & module invoicing
    DB-->>SIS: Return invoice_list.json & invoice_form.json
    SIS-->>Web: Return JSON View Schemas
    Web->>Web: DynamicModulePage.jsx renders Table + Columns + Actions
```

4.2.4 11.4 Cross-Module Integration Call Sequence Diagram

```
sequenceDiagram
    autonumber
    participant Inv as  invoicing Module
    participant Reg as  FunctionRegistry
    participant Contacts as  contacts_base Module

    Inv->>Reg: call("invoicing", "contacts_base", "get_contact", { contact_id: "..." })
    Reg->>Reg: Verify invoicing declared dependency in uses.json
    Reg->>Reg: Validate input_schema
    Reg->>Contacts: Dispatch to contacts_base.services.get_contact
    Contacts-->>Reg: Return { id: "...", display_name: "John Doe" }
    Reg->>Reg: Validate output_schema
    Reg-->>Inv: Return schema-validated contact payload
```

4.2.5 Command Line Render Test Code

```
# Command line to build docs site and verify mermaid syntax parsing
mkdocs build
```

4.3 12. Complete Worked Module Case Studies

4.3.1 12.1 Case Study 1: farms Module (Complete Files & Code Specs)

`modules/farms/manifest.json` :

```
{
  "name": "farms",
  "display_name": "Irrigation Farms",
  "version": "1.0.0",
  "module_type": "main",
  "target_app": "sis_web",
  "depends": [],
  "models": ["farm", "motor", "valve", "sensor"],
  "default_roles": ["farm_admin", "farmer", "viewer"],
  "description": "Core farm plot management, telemetry sensors, motor controls, and valve status."
}
```

`modules/farms/models/farm.json` :

```
{
  "name": "farm",
  "table_name": "mod_farms_farm",
  "label": "Farm Plot",
  "fields": [
    { "name": "name", "type": "string", "required": true, "unique": true, "label": "Farm Name" },
    { "name": "crop", "type": "string", "required": true, "label": "Crop Type" },
    { "name": "master_controller", "type": "string", "required": false, "label": "Controller ID" },
    { "name": "battery_pct", "type": "integer", "required": false, "default": 100, "label": "Battery %" },
    { "name": "power_grid", "type": "string", "required": false, "default": "Normal", "label": "Grid Status" }
  ]
}
```

4.3.2 12.2 Case Study 2: invoicing Module (Complete Files & Code Specs)

modules/invoicing/manifest.json :

```
{
  "name": "invoicing",
  "display_name": "Invoicing & Billing Management",
  "version": "1.0.0",
  "module_type": "main",
  "target_app": "sis_web",
  "depends": [],
  "models": ["invoice", "invoice_line"],
  "default_roles": ["invoicing_manager"],
  "requires_integration": ["users.get_user"],
  "exposes_integration": ["invoicing.get_invoice_total"],
  "description": "Industrial invoicing module with integration function capabilities."
}
```

modules/invoicing/models/invoice.json :

```
{
  "name": "invoice",
  "table_name": "mod_invoicing_invoice",
  "label": "Customer Invoice",
  "fields": [
    { "name": "number", "type": "string", "required": true, "unique": true, "label": "Invoice Number" },
    { "name": "customer_id", "type": "relation", "relation_target": "contacts_base.contact", "required": true, "label": "Customer Contact" },
    { "name": "total_amount", "type": "numeric", "required": true, "default": 0.00, "label": "Total Amount" },
    { "name": "status", "type": "string", "default": "draft", "label": "Status" }
  ]
}
```

modules/invoicing/integrations/uses.json :

```
{
  "requires": [
    { "module": "users", "function": "get_user" }
  ]
}
```

modules/invoicing/functions/exposes.json :

```
{
  "functions": [
    {
      "name": "get_invoice_total",
      "description": "Returns total calculated billing amount for invoice ID",
      "input_schema": { "invoice_id": "string" },
      "output_schema": { "invoice_id": "string", "total_amount": "numeric", "currency": "string" },
      "implementation": "invoicing.services.get_invoice_total"
    }
  ]
}
```

4.3.3 12.3 Case Study 3: schedule Module (Complete Files & Code Specs)

modules/schedule/manifest.json :

```
{
  "name": "schedule",
  "display_name": "Irrigation Scheduler",
  "version": "1.0.0",
  "module_type": "main",
  "target_app": "sis_web",
  "depends": ["farms"],
  "models": ["irrigation_job"],
  "default_roles": ["schedule_admin", "operator"],
  "description": "Automated irrigation scheduling bound to farms."
}
```

4.3.4 12.4 Case Study 4: pdf_exporter Extension Module (Complete Files & Code Specs)

modules/pdf_exporter/manifest.json :

```
{
  "name": "pdf_exporter",
  "display_name": "PDF Export Tools",
  "version": "1.0.0",
  "module_type": "extension",
  "target_app": "sis_web",
  "depends": [],
  "models": [],
  "ui_extensions": [
    {
      "target_module": "invoicing",
      "target_view": "view_invoice_list",
      "action_type": "button",
      "label": "Export PDF Invoice",
      "icon": "file-text"
    }
  ],
  "description": "Injects PDF export action button into Invoicing view screens."
}
```

4.3.5 Command Line Case Study Audit Code

```
# Command line to inspect all module manifest display names
Get-ChildItem -Path "modules" -Filter "manifest.json" -Recurse | ForEach-Object {
  $c = Get-Content $_.FullName | ConvertFrom-Json
  Write-Host "$($c.name) -> $($c.display_name) ($($c.module_type))"
}
```

5. Reference & Guides

5.1 13. REST API Endpoint Reference Guide

5.1.1 13.1 Module Installer & Management APIs

ENDPOINT PATH	HTTP METHOD	AUTH REQUIRED	DESCRIPTION
/api/modules	GET	Authenticated	Lists available and installed modules for user's account.
/api/modules/{module_name}/install	POST	Account Admin	Triggers 5-Engine installation pipeline for specified module.
/api/modules/{module_name}/uninstall	POST	Account Admin	Triggers module uninstallation and cleanup.
/api/modules/{module_name}/views	GET	Authenticated	Fetches JSON view schemas (<code>views/*.json</code>) for frontend rendering.

5.1.2 13.2 Data Access REST APIs

ENDPOINT PATH	HTTP METHOD	AUTH REQUIRED	DESCRIPTION
/api/data/{model_name}	GET	Authenticated	CRUD Read: Queries records from <code>mod_<module>_<model></code> with security checks.
/api/data/{model_name}	POST	Authenticated	CRUD Create: Inserts new record with security checks.
/api/data/{model_name}/{id}	PUT	Authenticated	CRUD Update: Updates existing record with security checks.
/api/data/{model_name}/{id}	DELETE	Authenticated	CRUD Delete: Soft-deletes record (<code>deleted_at = NOW()</code>) with security checks.

5.1.3 Command Line API Test Snippet

```
# Command line to test list available modules endpoint
Invoke-RestMethod -Uri "http://localhost:8001/api/modules" -Method Get

# Command line to fetch view schema for invoicing model
Invoke-RestMethod -Uri "http://localhost:8001/api/modules/invoicing/views" -Method Get
```

5.2 14. Developer Workflow, Testing & Troubleshooting

5.2.1 14.1 Authoring a New Module Step-by-Step

1. **Create Directory:** Create `modules/<my_module_name>/` .
2. **Author `manifest.json`:** Define `name` , `version` , `module_type: "main"` , and `models: [<"my_model">]` .
3. **Define Declarative Models:** Create `models/<my_model>.json` with field names, types (`string` , `numeric` , `relation`), namespace relation targets (`"other_mod.target"`), and constraints.
4. **Define Security Policies:** Create `security/access.json` declaring role CRUD permissions.
5. **Define UI Views:** Create `views/<my_model>_list.json` and `views/<my_model>_form.json` .
6. **Test Installation:** Issue API call to `/api/modules/<my_module_name>/install` .

5.2.2 14.2 Local Testing & Debugging Workflow

- **Database Inspection:** Verify table creation in database: `SELECT * FROM sys_model WHERE module_name = 'my_module';`
- **View Schema Verification:** Test view loading endpoint:
`curl -H "Authorization: Bearer <token>" http://localhost:8001/api/modules/my_module/views`
- **Container Log Monitoring:** Inspect container outputs: `docker compose logs -f`

5.2.3 14.3 Common Errors & Remediation Guide

ERROR MESSAGE / SYMPTOM	ROOT CAUSE	REMEDIATION STEP
Invalid identifier 'X': must match regex...	Malicious or malformed module field name/table name.	Ensure identifier contains only lowercase letters, numbers, and underscores (<code>^[a-z0-9_]{1,63}\$</code>).
Module 'X' requires missing dependencies: Y	Dependency module Y is not installed for account.	Install module Y prior to installing module X .
relation_target 'X' must be in 'module.model' format	Bare model name used without namespace.	Update relation_target to fully qualified format (e.g. <code>"contacts_base.contact"</code>).
Cannot link relation field 'A.b' -> 'C': target model not installed	Relation target model C does not exist in sys_model .	Ensure target module owning model C is listed in depends .
Permission Error: Module 'X' did not declare dependency...	Cross-module function call attempted without declaration in uses.json .	Add dependency declaration into integrations/uses.json .
SchemaValidationError: Missing required payload key	Inter-module function call payload failed JSON input/output schema check.	Fix call arguments to match exposed function input_schema .

5.2.4 Command Line Debugging Code

```
# Command line to check server API response status
Invoke-WebRequest -Uri "http://localhost:8001/health" | Select-Object StatusCode, StatusDescription
```

5.3 15. Quick Reference Cheatsheet & Terminal Commands

5.3.1 15.1 Docker Control Commands

```
# Command line to build and launch all Docker services in detached mode
docker compose up -d --build

# Command line to check running container status and health checks
docker compose ps

# Command line to inspect live combined logs from all containers
docker compose logs -f
```

5.3.2 15.2 Module Installer Pipeline Commands

```
# Command line to validate JSON files across module folders
Get-ChildItem -Path "modules" -Filter "*.json" -Recurse | ForEach-Object { Get-Content $_.FullName | ConvertFrom-Json }

# Command line to test health endpoint of host installer engine
Invoke-RestMethod -Uri "http://localhost:8001/health" -Method Get
```

5.3.3 15.3 API Curl / Powershell Test Commands

```
# Command line to test module listing endpoint for account
Invoke-RestMethod -Uri "http://localhost:8001/api/modules" -Method Get

# Command line to test installing farms module via REST API
$body = @({ account_id = "acc_1001" }) | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8001/api/modules/farms/install" -Method Post -Body $body -ContentType "application/json"

# Command line to fetch view schemas for farms module
Invoke-RestMethod -Uri "http://localhost:8001/api/modules/farms/views" -Method Get
```

6. Deep Dives & Gaps

6.1 16. Architecture & Security Critique: Gap Analysis

A review of this guide's architecture, installer engine, and security model, benchmarked against production plugin ecosystems (vTiger, Frappe/ERPNext).

6.1.1 16.1 What's Actually Solid

- **Declarative-only modules** (no arbitrary backend code shipped in a module) is the right call for a multi-tenant SaaS. It's the same philosophy Frappe uses with its "app" fixtures/hooks system, just pushed further toward pure JSON.
- **SQL identifier whitelisting** (`^[a-z0-9_]{1,63}$`) enforced before any DDL string interpolation is correct, and it's a check a lot of homegrown systems skip.
- **Two-phase FK resolution** (create relation columns as plain UUIDs first, apply FK constraints after all modules are loaded) solves a real, non-obvious ordering problem — most amateur module systems get this wrong or ignore it.
- **Transactional install with concurrency locking and rollback** is table stakes for a multi-tenant installer, but at least it's actually implemented here, not just described.
- **RBAC + ABAC split** (`role_model_access` , `role_field_access` , `role_record_rule`) roughly mirrors what vTiger calls "sharing rules" and field-level security — a reasonable shape for a permissions model.

6.1.2 16.2 Where It Falls Short of a Real Plugin System

1. **No provenance or integrity verification of a module before install:** No checksum, signature, or publisher identity attached to a module package.
2. **No permission/consent surface for admins:** No OAuth-style scope list shown to the installing admin before clicking Install.
3. **postMessage ("*")** **origin vulnerability:** Using `"*"` target origin in reference widgets defeats iframe sandboxing. Must validate `event.origin` .
4. **"Isolated process sandbox" for hooks is underspecified:** Timeout and memory limits are described, but the container primitive (gVisor, Firecracker, nsjail, Docker) is unstated.
5. **Audit logging is an afterthought:** `_record_audit_log` fires only on install failure. No runtime audit trail for queries or integration calls.
6. **No per-tenant resource quotas:** No limits on max tables per module, storage caps, or execution rate limits.
7. **domain_filter.operator** **missing explicit whitelist:** Needs explicit regex/enum validation before SQL interpolation.
8. **No secrets/credentials story:** No encrypted config or vault mechanism for external API keys.
9. **No uninstall data-retention policy:** Soft-delete applies to CRUD, but uninstall table retention policy is undefined.
10. **No compatibility gate between module and host platform version:** Missing `requires_platform_version` SemVer guard rail.

6.1.3 16.3 Priority Order to Close the Gaps

PRIORITY	GAP	WHY IT'S URGENT
1	<code>postMessage("*")</code> origin bug	Actively vulnerable pattern already in reference docs
2	<code>domain_filter.operator</code> whitelist	Potential SQL injection path, inconsistent with rest of spec's rigor
3	Module provenance/signing	Foundational — everything else assumes you trust the module folder
4	Named sandbox mechanism for hooks	Current isolation claims aren't verifiable without this
5	Runtime audit trail (not just install-failure logging)	Needed for incident response and compliance
6	Per-tenant resource quotas	Prevents a single module from degrading the whole platform
7	Install-time permission/consent screen	Admin trust and transparency
8	Secrets management channel	Currently no sanctioned way for a module to hold a credential
9	Uninstall data-retention policy	Operational gap, will surface in production incidents
10	Platform/module version compatibility gate	Prevents silent breakage on platform upgrades

6.1.4 17. Installer Engine: Internal Bugs & Lifecycle Completeness Gaps

- **Foreign key failures are silently swallowed:** Failed FK constraints log warnings instead of stopping install.
- **Exposed-function schema checks only verify key presence, not type:** Schema validator checks key exists, not that type matches.
- **Exposed function implementations run unsandboxed, in-process:** Functions execute directly in host process without memory/timeout limits.
- **Successful installs are never audit-logged:** Only failure branch writes to audit log.
- **Cache invalidation happens outside error handling:** Occurs outside try/finally block after lock release.
- **Missing lifecycle operations:** No fully specified `uninstall` sequence or `upgrade` migration caller. No installer idempotency check.
- **Installed-module state table schema is missing:** Storage schema for `sys_installed_module` is unstated.

6.1.5 18. Missing Engine: Runtime Access Enforcement (`chokepoint.py`)

- `security/access.json` declares rules; it does not enforce them.
- Every call to `/api/data/*` requires runtime enforcement:
- Resolve calling user role for account.
- Check `role_model_access` for CRUD permission.
- Check `role_field_access` to build column allow-lists for read/write.
- Check `role_record_rule` to append `domain_filter` SQL conditions alongside `account_id`.
- Execute query safely.

6.1.6 19. Missing Link: Role Ownership vs Platform User Management

Recommended Fix: Modules Declare Permission Points, Never Roles

1. **Modules declare models, fields, and features:** `security/access.json` becomes a **permission catalog file** listing available capabilities.
 2. **Engine 4 registers permission points into platform catalog:** Platform role system creates roles and assigns users to them.
 3. **Resolves authority conflicts, assignment APIs, and role-name collisions.**
-

6.1.7 Command Line Gap Analysis Check Code

```
# Command line to search codebase for chokepoint or enforcement modules
Get-ChildItem -Path "." -Filter "*chokepoint*" -Recurse
```